# 202322567

# Koetje Khothalang

# Cs4433 Week 1

# PROJECT PLANNING AND MANAGEMENT

- ## Feasibility Study

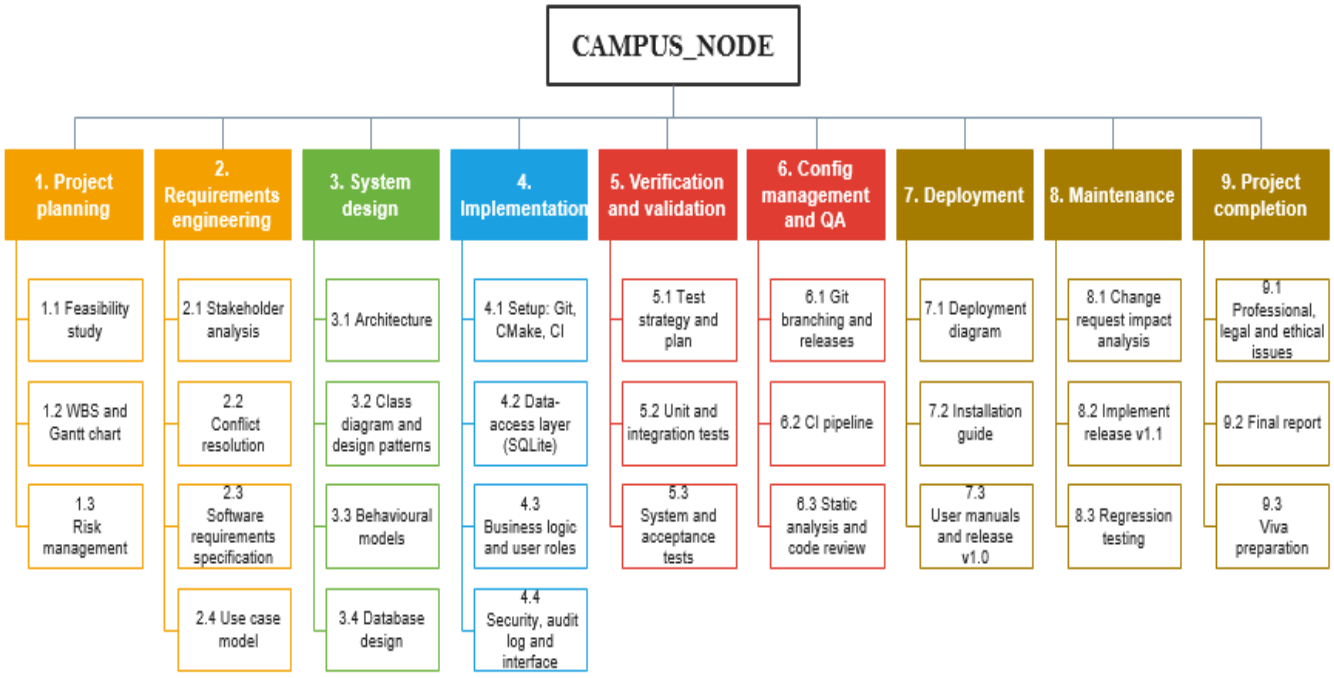
1. **Technical Feasibility**: The proposed system is technically feasible because all functionalities can be catered for using the specified technologies (C++ and SQ Lite). These technologies are generally free and need no servers and can work offline. SQLite also copes easily with 800 Learner records, a client-server database (MySQL) was considered and rejected because it needs a server to be installed and maintained on old school computers which introduces more complexities than necessary. The main technical risk is performance with many users stored, this will be reduced by indexing key tables, testing with 800 generated record against a set time.
2. **Operational Feasibility**: Feasible because the system is hosted in the school's environment and supports what the four user groups already do. The Administrator manages accounts, lecturer sets and marks assessments, HOD moderates marks and students submit work and view feedback. However, the principal stated that teachers are not IT experts so the system's interface must be user friendly and menu driven. The stakeholder conflicts such as when students see the marks must be resolved before implementation.
3. **Economic Feasibility:** Yes, system is economically feasible because all tools (C++17, SQLite, CMake, Git, Google Test) are free or open source. It runs on the school's existing computers, and it need no internet connection, so there are no license or hosting costs. The main cost is development time, stretched over a period of 7 weeks. The benefits are less paper-based marking, a clear record of every mark change, and faster feedback to students. Features that would raise cost, such as online payment and video conferencing, are out of scope.
4. **Legal Feasibility:** The system is legally feasible if learners' personal information and assessment records are properly protected. Most learners are minors, so data protection matters even more. The relevant law is Lesotho's Data Protection Act, 2011 (Act No. 5 of 2012). The design responds to this in three ways: passwords are stored as salted hashes, access is limited by role so only the right people see restricted data, and every mark change is recorded in an audit log. The open-source tools used allow free use.

5. **Schedule Feasibility:** The project can be completed in seven weeks, but the schedule is tight because it is a one-person project. The work is divided into phases with set time limits and three review gates. If time runs short, all *Must* requirements will be completed first and *should* and *could* features dropped. The Gantt chart and risk register are used to track progress

   **RECOMMENDATION: proceed with the project.** The system is feasible in all five areas. The tools are free and suit old computers, the offline requirement is met by SQLite, and the cost is only development time. The two main conditions for success are resolving the stakeholder conflicts before design starts and keeping the scope within the *Must* requirements.

- ## Work Breakdown and Schedule

  a) Work Breakdown Structure



b) Gantt Chart

Critical path

The critical path is the longest chain of dependent tasks in the project. Any delay to a task on this path delays the whole project because those tasks have no slack, meaning if any task slips by a day then every other dependent task will be delayed by a day.

In this project it is: 1 → 2 → 3 → 4 → 5 → Gate 2 → 6 → 7 → 9 → 11 → 12, and it uses all seven weeks (shown in red on the Gantt chart). Each task needs the one before it to be finished. For example, design must pass Gate 2 before coding can start, system testing (9) needs the code (7) to be finished, and the change request (12) can only begin after release v1.0 (11). The blue tasks have slack, so they can move without delaying anything else. These include the test plan and unit tests (8), which can start as soon as the setup is done, the static analysis and code review (10), and the report writing and viva preparation (13), which can be drafted during earlier weeks.

## CAMPUSNODE GANTT CHART (7 WEEKS)

| ID | Task | Dependency | Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | Feasibility, WBS, risk register | - | | | | | | | |
| 2 | Stakeholder analysis and conflicts | 1 | | | | | | | |
| 3 | SRS and use case model | 2 | | | | | | | |
| G1 | **Gate 1: requirements review** | 3 | | | | | | | |
| 4 | Architecture and class diagram | 3 | | | | | | | |
| 5 | Behaviour models and database design | 4 | | | | | | | |
| G2 | **Gate 2: design review** | 5 | | | | | | | |
| 6 | Setup (Git, CMake, CI), data layer | G2 | | | | | | | |
| 7 | Business logic, UI, security, audit log | 6 | | | | | | | |
| 8 | Test plan, unit and integration tests | 6 | | | | | | | |
| 9 | System and acceptance tests, defect log | 7, 8 | | | | | | | |
| 10 | Static analysis and code review | 7 | | | | | | | |
| 11 | Deployment docs, release v1.0 | 9 | | | | | | | |
| G3 | **Gate 3: V&V review** | 9, 11 | | | | | | | |
| 12 | CR1 analysis, release v1.1, regression | 11 | | | | | | | |
| 13 | Ethics section, final report, viva prep | 11 | | | | | | | |

Critical path (no slack) | Task with slack | Milestone / phase gate

**Critical path: 1 > 2 > 3 > 4 > 5 > G2 > 6 > 7 > 9 > 11 > 12**

- **Risk Management**

A risk is something that might go wrong and hinder the project. A Risk Register lists each risk, scores how serious it is, with the scores changing at every phase gate as the project moves. Also records what to do about each risk and who is responsible. The risks were identified from the case study, the stakeholder statements and the project's constraints: a single developer, a seven-week schedule, and old computers that work offline. Each risk is classed as Technical, Project or People.

Likelihood and impact are each scored from 1 to 5, using the scales in Tables 1 and 2. $Exposure = Likelihood \times impact$**.** An exposure of 12 or more is rated **High** (act immediately and check weekly), 6 to 11 is **Medium** (monitor at each gate) and 5 or less is **Low** (accept and watch). These scales and thresholds are my own choice, suited to a small seven-week project

Table 1: Likelihood Scale

| SCORE | MEANING | CHANCE OF HAPPENING |
|---|---|---|
| 1 | Rare | Under 10% |
| 2 | Unlikely | 10% to 30% |
| 3 | Possible | 30% to 50% |
| 4 | Likely | 50% to 80% |
| 5 | Almost Certain | Over 80% |

Table 2: Impact Scale

| SCORE | MEANING | EFFECT ON PROJECT |
|---|---|---|
| 1 | Negligible | Under half a day lost, no effect on deliverables |
| 2 | Minor | About one day lost, easily absorbed |
| 3 | Moderate | Several days lost or one deliverable noticeably weaker |
| 4 | Major | About a week lost, or a Must requirement at risk. |
| 5 | Severe | Work lost, a gate or final deadline missed OR project fails. |

The register holds 10 risks: four Technical, four Project and two People. Four are rated High: scope creep (R1, exposure 16), falling behind schedule or illness (R2, 15), unresolved stakeholder conflicts (R3, 12) and not enough time for testing (R9, 12). They are all project or people risks which is expected for a one person project with a fixed deadline. The technical risks

are Medium because SQLite, C++17 and recognized password hashing libraries are well documented.

The register is reviewed at each phase gate (end of Weeks 2, 3 and 6). Each risk is re-scored, its status is updated (Open, Reduced, Closed or Occurred) and any new risks will be added.

Each mitigation acts early because a risk is cheaper to prevent than to fix.

Table 3: Risk Register

| Risk Register: Campus Node | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| ID | Risk description | Category | Likelihood (1-5) | Impact (1-5) | Exposure (L x I) | Rating | Mitigation | Owner | Status |
| R1 | Scope creep: adding features beyond the Must requirements (or out-of-scope items such as plagiarism checks) uses up time needed for testing and documentation. | Project | 4 | 4 | 16 | High | Freeze the scope using MoSCoW at Gate 1. Build Must items first. Log new ideas as Could or Won't. Check the brief's out-of-scope list before adding anything. | Developer (me) | Open |
| R2 | Falling behind schedule or illness: as the only team member, any lost week delays every later phase. | Project | 3 | 5 | 15 | High | Compare progress with the Gantt chart every week. Start tasks early. Keep Could items optional so they can be dropped. Keep one unplanned day per week as a buffer. | Developer (me) | Open |
| R3 | Unresolved stakeholder conflicts: for example the Lecturer wants instant marks but the Principal wants HoD approval first, which leads to an unstable SRS and rework later. | People | 4 | 3 | 12 | High | List every conflict and its resolution in the stakeholder analysis. Apply the HoD-moderation rule from the case study. Get the supervisor to confirm the resolutions at Gate 1. | Developer (me) | Open |
| R4 | Security flaw: weak password storage or SQL injection exposes learner data and marks. | Technical | 2 | 5 | 10 | Medium | Store salted hashes using a recognised library (e.g. libsodium with Argon2). Use parameterised queries only. Run a static analyser. Test role permissions. | Developer (me) | Open |
| R5 | Loss of code or documents: laptop failure or accidental deletion destroys work. | Project | 2 | 5 | 10 | Medium | Commit to a hosted Git repository at least daily. Keep a weekly backup of documents in a second location. | Developer (me) | Open |
| R6 | Learning curve: unfamiliar tools (C++17 features, SQLite, GoogleTest, CI) slow development. | Technical | 3 | 3 | 9 | Medium | Build small prototypes in Weeks 1 to 3, before the design is frozen. Use official tutorials. Choose well-documented | Developer (me) | Open |
| R7 | Stakeholders unavailable: stakeholders or the supervisor cannot answer questions or review at the gates. | People | 3 | 3 | 9 | Medium | Book the gate reviews early. Use the case-study statements as the stakeholder source. Record assumptions in the SRS and confirm them at the next gate. | Supervisor | Open |
| R8 | Poor performance with 800 students: slow login or list screens on old computers. | Technical | 2 | 4 | 8 | Medium | Index key columns. Test with 800 generated users. Set a measurable response target in the SRS (e.g. login within 2 seconds). | Developer (me) | Open |
| R9 | Not enough time for testing: implementation overruns and verification is squeezed into Week 6. | Project | 3 | 4 | 12 | High | Write unit tests while coding each module. Set up CI in Week 4. Cut Should and Could features before cutting tests. | Developer (me) | Open |
| R10 | Database problems: SQLite file corruption or a bad schema change causes loss of marks or audit-log data. | Technical | 2 | 4 | 8 | Medium | Use transactions for mark changes. Keep schema scripts in version control. Add a simple database backup step to the installation guide. | Developer (me) | Open |